

JavaScript: Funciones como "First-Class Citizens"

Guía docente para explicar por qué las funciones son ciudadanas de primera clase.

En JavaScript, las funciones son consideradas **First-Class Citizens** (ciudadanas de primera clase) porque reciben el mismo trato que cualquier otra variable del lenguaje. Esto significa que una función puede ser asignada a una variable, ser pasada como argumento a otra función o ser retornada por otra función.

1. Asignar una función a una variable (Function Expression)

Muestra a los alumnos que una función puede guardarse dentro de una variable de la misma manera que guardamos un número o un texto.

```
// Asignamos una función anónima a la variable 'saludar'
const saludar = function(nombre) {
  return "Hola, " + nombre + "!";
};

// Se ejecuta llamando a la variable como si fuera la función
console.log(saludar("Carlos")); // Resultado: Hola, Carlos!
```

2. Pasar una función como argumento (Callback)

Demuestra cómo una función puede recibir a otra por parámetro para delegar un comportamiento. Este es el fundamento básico de los métodos tradicionales y la programación asíncrona.

```
function procesarEntrada(callback) {
  const nombre = "Ana";
  // Ejecutamos la función que recibimos por parámetro
  return callback(nombre);
}

// Creamos una función normal para transformar el texto
function enMayusculas(texto) {
  return texto.toUpperCase();
}

// Pasamos la función 'enMayusculas' como argumento
console.log(procesarEntrada(enMayusculas)); // Resultado: ANA
```

3. Retornar una función desde otra función

Ideal para introducir la idea de que una función puede fabricar y devolver otra función configurada con un comportamiento específico.

```
function crearMultiplicador(factor) {  
  // Retornamos una nueva función tradicional que recuerda el 'factor'  
  return function(numero) {  
    return numero * factor;  
  };  
}  
  
const duplicar = crearMultiplicador(2);  
const triplicar = crearMultiplicador(3);  
  
console.log(duplicar(5)); // Resultado: 10  
console.log(triplicar(5)); // Resultado: 15
```